Лабораторная работа 1

Параллельное численное интегрирование методом правых прямоугольников

Две реализации на C++: POSIX Threads и std::thread

Параметр	Значение
Выполнили	Виткулевский Т. В. Волков К. В.
Группа	АВТ-412
Преподаватели	Ландовский В. В., доцент, к. т. н. кафедры АСУ; Дворецкая В. К., ассистент кафедры АСУ
Вариант	Правые прямоугольники, $f(x) = (1+x^2)/(x^3\sqrt{}(2+x))$, [2;5]
Дата эксперимента	20 октября 2026 года
Основной запуск	N = 20,000,000; p = 8; p_max = 16; 7 повторов

Цель работы — освоить численное интегрирование, обоснованный выбор шага и оценку погрешности, реализовать параллельное вычисление двумя средствами C++ и исследовать масштабируемость.

Получено I_N = 0.450968532781073; аналитический эталон I = 0.4509685503223266. Абсолютная ошибка составляет 1.75413e-08. Оба API дают одинаковый результат в пределах округления.

1 Постановка задачи и численный метод

Вычисляется определённый интеграл $I = \int_2^5 (1+x^2)/(x^3\sqrt{(2+x)})\,dx$. Функция непрерывна и положительна на всём отрезке, поэтому интеграл совпадает с площадью под её графиком.

Формула правых прямоугольников

Отрезок разбивается на N равных частей: $h = (b-a)/N = 3/N$. На каждом подотрезке высота прямоугольника берётся в правом конце: $x_i = a+ih$, $i = 1,\ldots,N$.

```
I_N = h * sum(f(a + i*h), i=1..N)
В коде: i=0..N-1, узел x = a + (i+1)*h
```

Функция убывает, поэтому правые прямоугольники занижают интеграл. Параллелизация меняет порядок суммирования, но не сетку узлов и не формулу метода.

Сопоставление погрешностей методов задания

Метод	Порядок	Граница $\lvert I-I_N\rvert$	Ограничение N
Левые / правые прямоугольники	O(h)	$L\,h\,M_1 / 2$	N ≥ 1
Средние прямоугольники	$O(h^2)$	$L\,h^2\,M_2 / 24$	N ≥ 1
Трапеции	$O(h^2)$	$L\,h^2\,M_2 / 12$	N ≥ 1
Симпсон	$O(h^4)$	$L\,h^4\,M_4 / 180$	N чётное
Ньютон–Котес 3/8	$O(h^4)$	$L\,h^4\,M_4 / 80$	N кратно 3

Здесь $L = b-a$, $M_k = \max |f^{(k)}(x)|$ на $[a;b]$; для соответствующей границы нужна непрерывная производная указанного порядка. В строке Ньютона–Котеса рассматривается составная формула 3/8 с интерполяционным полиномом третьей степени. В программе по варианту реализованы только правые прямоугольники.

2 Шаг интегрирования и оценка погрешности

Представим f(x) = (1/x + 1/x^3) / √(x+2). Тогда производная имеет вид:

```
f'(x) = -(x^-2 + 3*x^-4)/sqrt(x+2)
        -(x^-1 + x^-3)/(2*(x+2)^(3/2))
```

Каждое слагаемое в |f′(x)| положительно и убывает при x > 0. Максимум на [2;5] достигается при x = 2: M_1 = 33/128 = 0,2578125.

```
|I - I_N| <= (b-a)*h*M1/2 = 297/(256*N) = 1.16015625/N
N >= ceil(1.16015625/eps),  h = 3/N
```

Требуемая ε	Достаточное N	Шаг h	Граница ошибки
1e-03	1161	2.583979e-03	9.992733e-04
1e-05	116016	2.585850e-05	9.999968e-06
1e-07	11601563	2.585858e-07	1.000000e-07

Для эксперимента N = 20,000,000, h = 1.500000e-07, теоретическая граница равна 5.80078125e-08. Это оценка ошибки квадратуры в точной арифметике; при очень малом шаге дополнительно действует округление double.

Аналитический эталон и независимая проверка

Подстановка t = √(x+2) сводит интеграл к рациональной функции 2/(t^2−2) + 2/(t^2−2)^3. Первообразная:

```
F(x) = 35/(32*sqrt(2))*ln((sqrt(x+2)-sqrt(2))/(sqrt(x+2)+sqrt(2)))
     + sqrt(x+2)*(3/(16*x) - 1/(4*x*x))
I = F(5) - F(2)
```

Эталон, вычисленный Decimal с точностью 60 значащих цифр:

```
0.450968550322326660168633675195179549577893014459770257519547
```

Независимая составная формула Симпсона на 16 384 частях в арифметике Decimal отличается от аналитического эталона на 5.030e-18. Это контроль эталона, а не замена метода варианта.

Оценка Рунге

Порядок метода q = 1. Для уточнённого результата I_{2N} оценка Рунге равна $|I_{2N}-I_N|/(2^1-1) = |I_{2N}-I_N|$; для грубого I_N асимптотическая оценка вдвое больше. Наблюдаемый порядок вычисляется как $\log_2(|I-I_N|/|I-I_{2N}|)$.

3 Диаграмма деятельности

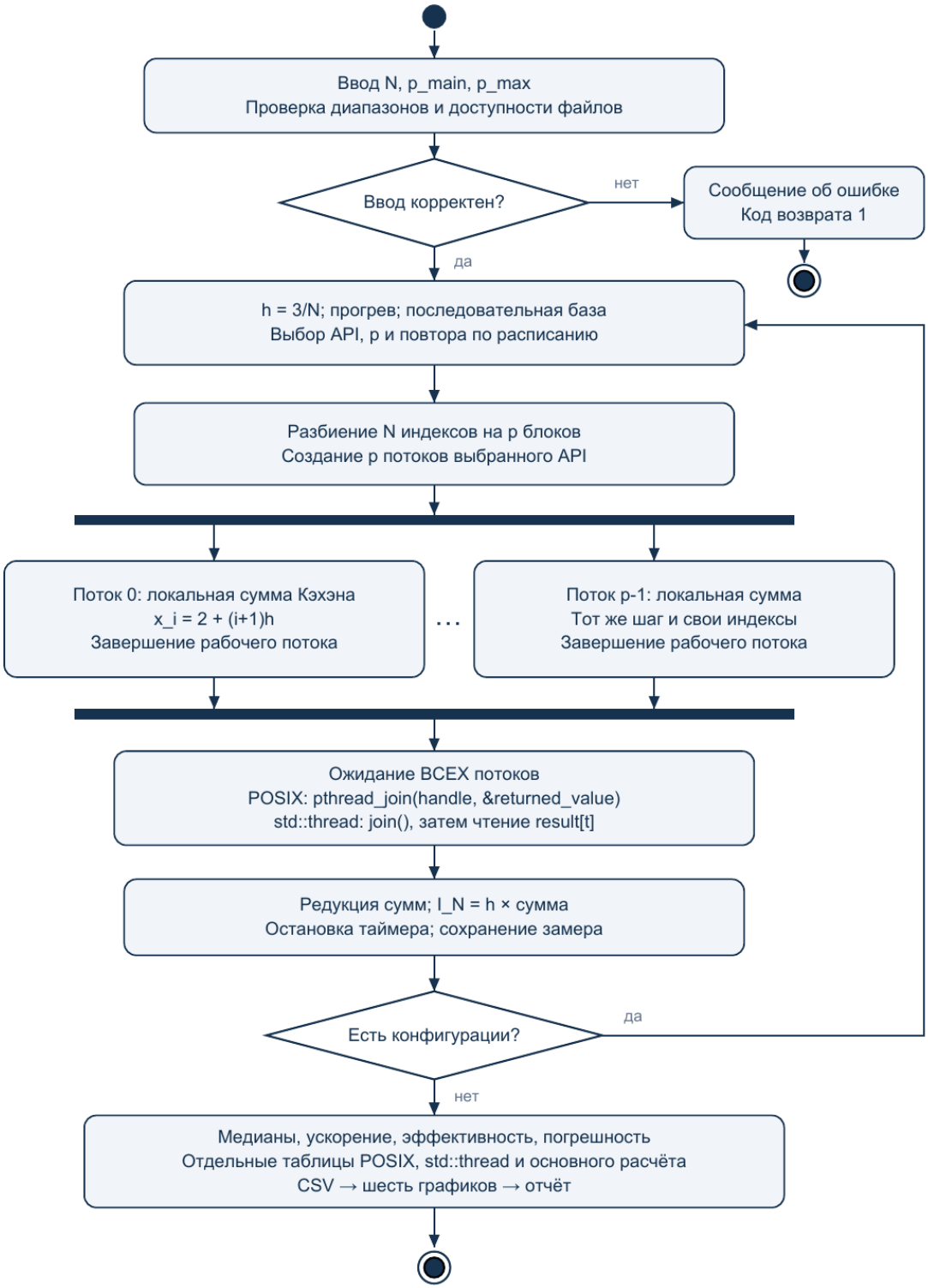


Рисунок 1. Алгоритм одного эксперимента. Вилкой и объединением показаны рабочие потоки выбранного API; технологии измеряются по очереди. Ошибки создания обрабатываются с ожиданием уже запущенных потоков.

4 Организация программы

Распределение вычислений

Для p потоков q = N div p, r = N mod p. Поток t получает начало l_t = tq + min(t,r) и конец u_t = l_t + q + [t < r]; индексы обрабатываются на полуинтервале [l_t,u_t). Соседние блоки стыкуются без пропусков и перекрытий, а их размеры различаются не более чем на единицу.

POSIX Threads

pthread_create передаёт потоку структуру с границами блока и шагом. Рабочая функция выделяет double в куче, записывает локальную сумму и возвращает указатель; основной поток получает именно возвращённое значение вызовом pthread_join(handle, &raw). После чтения unique_ptr освобождает память; указатель на локальную переменную стека не возвращается.

std thread

Функция worker(first, last, a, h, double& result) получает параметры отдельно. В конструктор std::thread передаются first, last, A, h и std::ref(results[t]); структура параметров не используется. Каждый поток пишет только в свой элемент заранее выделенного массива, а чтение происходит после join, поэтому гонки данных нет.

Точность и обработка ошибок

Локальные суммы и окончательная редукция используют суммирование Кэхэна. При частичном сбое создания потоков уже запущенные потоки завершаются до освобождения их аргументов; неуспешный pthread_join приводит к диагностике и остановке вместо обращения к недействительным данным.

Файл	Назначение
integration.hpp / integration.cpp	Интерфейс, функция варианта, сетка, последовательная сумма, эталон
posix.cpp	POSIX-реализация и получение результата через pthread_join
std_thread.cpp	Реализация std::thread с отдельными аргументами
main.cpp	Ввод, проверка, измерения, таблицы, экспорт CSV, самопроверка

Использована предоставленная Pthreads-win32 2.9.1, pthreadVC2.lib и pthreadVC2.dll x64. Для современного MSVC определено HAVE_STRUCT_TIMESPEC после подключения time.h, чтобы старый pthread.h не переопределял timespec из UCRT.

5 Методика эксперимента и проверка

Параметр	Значение
Процессор	12th Gen Intel(R) Core(TM) i5-12450H
ОС и число логических потоков	Windows x64; 12
Компилятор и сборка	MSVC 14.51.36231, Visual Studio 18; Release x64, оптимизация /O2
Число разбиений	20,000,000
Основное / максимальное число потоков	8 / 16
Повторы и прогрев	7 повторов; один прогрев каждой конфигурации
Таймер и статистика	std::chrono::steady_clock; медиана
Порядок измерений	API и p перемешиваются между повторами, seed 412

Измерение охватывает выделение рабочих массивов, создание потоков, вычисление, join, редукцию и освобождение временных ресурсов. Консольный вывод, запись CSV, прогрев и вычисление погрешности вынесены за измеряемый участок. Однопоточная база каждого API также создаёт один рабочий поток; прямой последовательный цикл измерен отдельно.

```
S_API(p) = T_API(1) / T_API(p)
E_API(p) = S_API(p) / p * 100%
S_ideal(p) = p
S_direct(p) = T_direct / T_API(p)
```

Короткие измерения зависят от планировщика, фоновой нагрузки и частоты процессора. Небольшое превышение 100 % на отдельных конфигурациях отражает отличие условий исполнения от однопоточной базы и не доказывает устойчивое сверхлинейное масштабирование.

Проверки корректности

Встроенный --self-test проверяет N = 1, 2, 7, 31, 1000, 10000 и допустимые p до 8: покрытие всех индексов, совпадение обеих технологий с последовательной суммой, границу ошибки и первый порядок сходимости. Отдельный Python-скрипт подтвердил медианы, ускорения и эффективность всех 32 строк таблицы по 231 исходному замеру, а также отклонение девяти некорректных вариантов командной строки.

Максимальный размах значений интеграла между всеми записанными запусками: 5.551e-17. Это на много порядков меньше ошибки дискретизации; итог всех проверок — PASS.

6 Основной результат и исследование точности

Режим	p	Время, с	Интеграл	Tseq / Tp
direct	1	0.079467800	0.450968532781073	1.0000
POSIX	8	0.021008500	0.450968532781073	3.7826
std::thread	8	0.021158000	0.450968532781073	3.7559

В этой таблице основание ускорения — прямой последовательный цикл. В сводных таблицах следующего раздела основание — один рабочий поток соответствующей технологии; поэтому значения ускорений отличаются.

Абсолютная погрешность и сходимость

N	h	I_N	$\|I-I_N\|$	Порядок
10	3.00e-01	0.417641634018	3.3327e-02	0.96263
100	3.00e-02	0.447477943389	3.4906e-03	0.99636
1000	3.00e-03	0.450617901686	3.5065e-04	0.99964
10000	3.00e-04	0.450933469578	3.5081e-05	0.99996
100000	3.00e-05	0.450965042089	3.5082e-06	1.00000
1000000	3.00e-06	0.450968199497	3.5082e-07	1.00000
20000000	1.50e-07	0.450968532781	1.7541e-08	1.00000

Оценка Рунге и теоретическая граница

N	Рунге для I_{2N}	Граница для I_N	Относительная ошибка I_N
10	1.62262e-02	1.16016e-01	7.39008e-02
100	1.74089e-03	1.16016e-02	7.74024e-03
1000	1.75280e-04	1.16016e-03	7.77546e-04
10000	1.75399e-05	1.16016e-04	7.77898e-05
100000	1.75411e-06	1.16016e-05	7.77933e-06
1000000	1.75412e-07	1.16016e-06	7.77937e-07
20000000	8.77063e-09	5.80078e-08	3.88968e-08

С ростом N ошибка убывает, а наблюдаемый порядок приближается к единице. Отрицательный знак $I_N - I$ согласуется с убыванием функции; заданная теоретическая граница выполнена для каждой проверенной сетки.

7 Сводные таблицы результатов

POSIX

Потоки	Время, с	Результат	Ускорение	Эффект., %
1	0.080652900	0.450968532781073	1.0000	100.00
2	0.040158300	0.450968532781073	2.0084	100.42
3	0.027054300	0.450968532781073	2.9811	99.37
4	0.021490400	0.450968532781073	3.7530	93.82
5	0.030932500	0.450968532781073	2.6074	52.15
6	0.021867600	0.450968532781073	3.6882	61.47
7	0.018835600	0.450968532781073	4.2819	61.17
8	0.021008500	0.450968532781073	3.8391	47.99
9	0.020278200	0.450968532781073	3.9773	44.19
10	0.018785200	0.450968532781073	4.2934	42.93
11	0.019698000	0.450968532781073	4.0945	37.22
12	0.017410200	0.450968532781073	4.6325	38.60
13	0.019271800	0.450968532781073	4.1850	32.19
14	0.018047000	0.450968532781073	4.4690	31.92
15	0.019723500	0.450968532781073	4.0892	27.26
16	0.019088000	0.450968532781073	4.2253	26.41

Ускорение рассчитано относительно одного рабочего потока POSIX. Данные — медианы семи повторов; все исходные времена сохранены в results/raw.csv.

7 Сводные таблицы результатов продолжение

std::thread

Потоки	Время, с	Результат	Ускорение	Эффект., %
1	0.084935700	0.450968532781073	1.0000	100.00
2	0.041590000	0.450968532781073	2.0422	102.11
3	0.027684100	0.450968532781073	3.0680	102.27
4	0.022156100	0.450968532781073	3.8335	95.84
5	0.025214600	0.450968532781073	3.3685	67.37
6	0.023120400	0.450968532781073	3.6736	61.23
7	0.019897000	0.450968532781073	4.2688	60.98
8	0.021158000	0.450968532781073	4.0144	50.18
9	0.019706200	0.450968532781073	4.3101	47.89
10	0.017189600	0.450968532781073	4.9411	49.41
11	0.019558900	0.450968532781073	4.3426	39.48
12	0.017931700	0.450968532781073	4.7366	39.47
13	0.018959800	0.450968532781073	4.4798	34.46
14	0.019339400	0.450968532781073	4.3918	31.37
15	0.018025900	0.450968532781073	4.7119	31.41
16	0.019138900	0.450968532781073	4.4379	27.74

Ускорение рассчитано относительно одного рабочего потока std::thread. Данные — медианы семи повторов; все исходные времена сохранены в results/raw.csv.

8 Разброс измерений и идеальная эффективность

p	POSIX Q1–Q3, мс	std Q1–Q3, мс	POSIX S/p	std S/p
1	80.504 – 82.701	83.231 – 91.306	1.0000	1.0000
2	39.893 – 40.410	40.127 – 41.899	1.0042	1.0211
3	26.897 – 29.059	27.193 – 29.792	0.9937	1.0227
4	20.463 – 27.830	21.902 – 22.749	0.9382	0.9584
5	24.971 – 32.873	24.615 – 31.433	0.5215	0.6737
6	21.025 – 29.720	21.267 – 28.113	0.6147	0.6123
7	18.568 – 22.397	19.283 – 22.067	0.6117	0.6098
8	19.058 – 21.564	19.195 – 22.990	0.4799	0.5018
9	19.313 – 24.198	19.093 – 23.514	0.4419	0.4789
10	17.908 – 21.957	17.011 – 21.738	0.4293	0.4941
11	18.137 – 21.677	17.979 – 24.035	0.3722	0.3948
12	15.668 – 19.932	16.415 – 18.402	0.3860	0.3947
13	18.180 – 20.600	17.575 – 19.224	0.3219	0.3446
14	17.295 – 18.732	18.278 – 23.030	0.3192	0.3137
15	18.353 – 21.722	17.430 – 20.851	0.2726	0.3141
16	17.356 – 23.113	17.477 – 20.222	0.2641	0.2774

Q1–Q3 показывает центральную половину замеров и помогает оценить устойчивость сравнения. Доля S/p равна эффективности, делённой на 100; идеал равен 1. Оптимум по минимальной медиане относится к данному запуску, а не является универсальной настройкой компьютера.

Технология	Минимум T(p), с	p при минимуме	S(p)	E(p), %
POSIX	0.017410200	12	4.6325	38.60
std::thread	0.017189600	10	4.9411	49.41

9 Графики эксперимента

9.1 Время выполнения

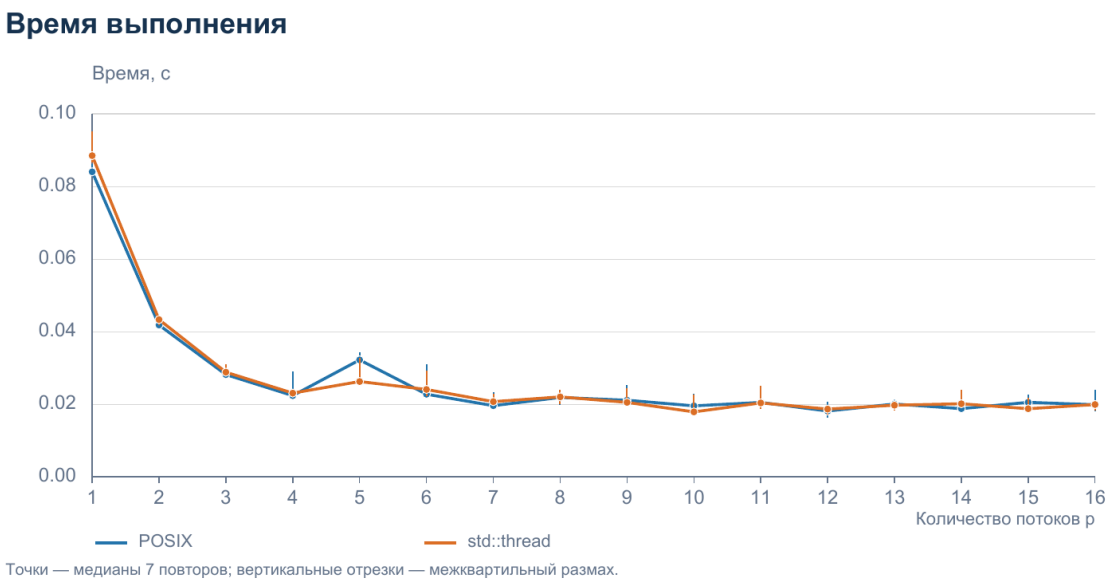


Рисунок 2. Время выполнения.

Время резко уменьшается при переходе от одного к нескольким потокам. Минимумы медиан: POSIX — 0.017410 с при p = 12; std::thread — 0.017190 с при p = 10. Неровности после p = 4 согласуются с влиянием распределения нагрузки и фоновых процессов; по этим данным нельзя выделить единственную причину.

9.2 Ускорение

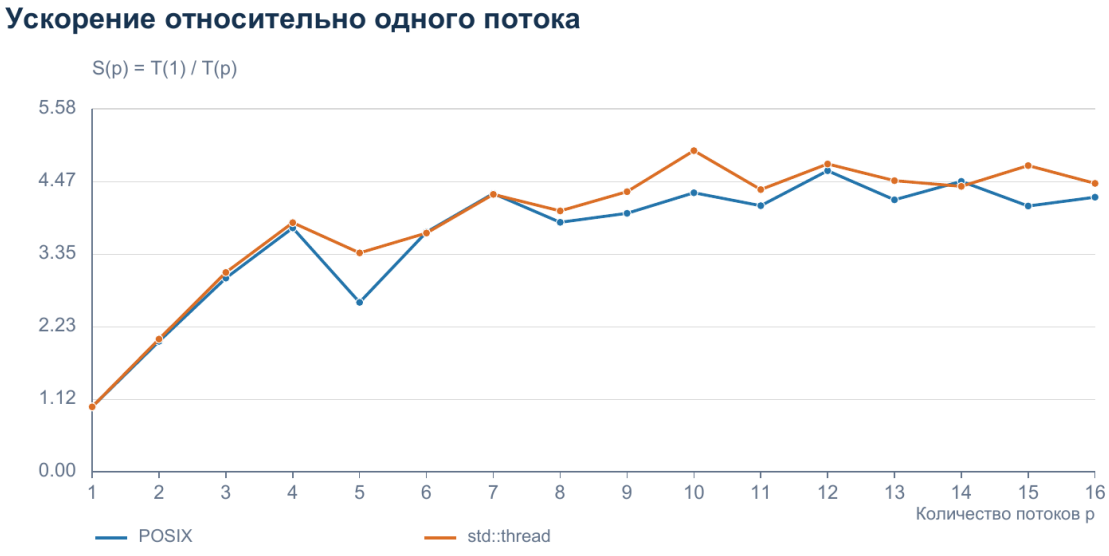


Рисунок 3. Ускорение.

Максимальное измеренное ускорение POSIX равно 4.633, std::thread — 4.941. Ускорение вычислено по собственной однопоточной базе каждого API, поэтому сравнивать технологии по абсолютному времени корректнее, чем только по S(p).

9 Графики эксперимента продолжение

9.3 Эффективность использования потоков

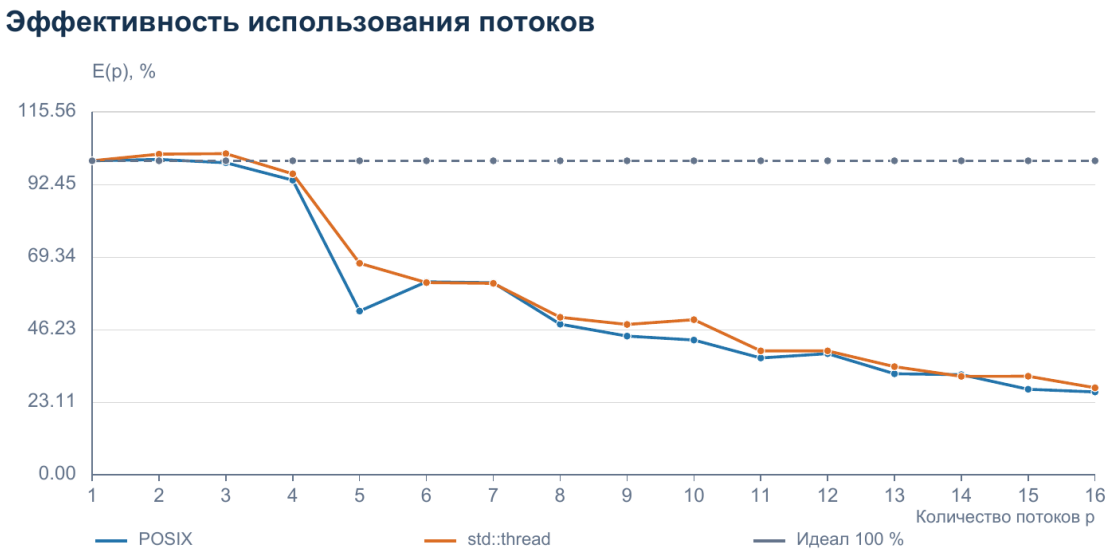


Рисунок 4. Эффективность использования потоков.

Эффективность близка к 100 % на малом числе потоков и заметно падает при дальнейшем увеличении p. Это означает, что добавление потоков даёт всё меньшую отдачу на один поток; отдельные значения немного выше 100 % объясняются изменчивостью измерений и условий исполнения.

9.4 Результаты вычислений

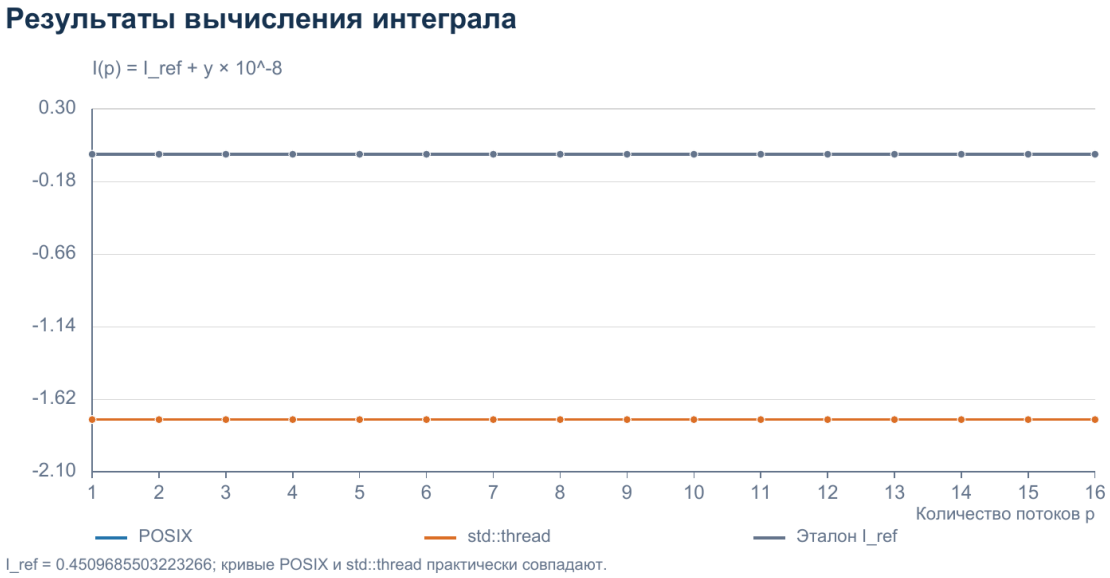


Рисунок 5. Результаты вычислений.

Для всех p используется одна и та же сетка. На графике показано $I_N = I_ref + y \cdot 10^{-8}$, чтобы наглядно видеть общую ошибку относительно эталона; две технологические кривые практически полностью совпадают. Максимальное различие между записанными результатами равно $5{,}55 \cdot 10^{-17}$.

9 Графики эксперимента продолжение

9.5 Идеальное и фактическое ускорение

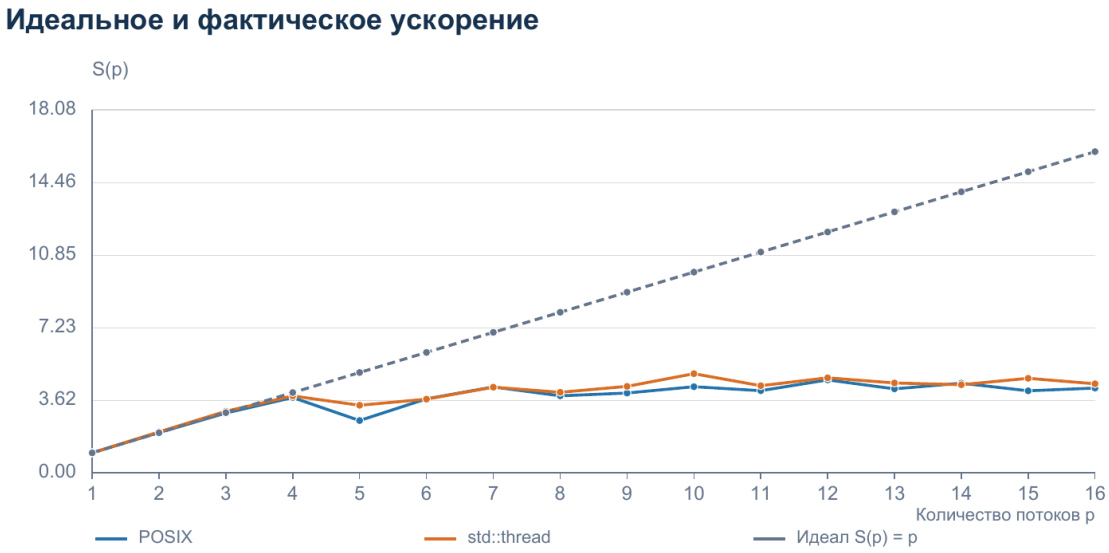


Рисунок 6. Идеальное и фактическое ускорение.

Идеальное ускорение S = p растёт линейно, а фактическое выходит на область насыщения около 4–5. Создание потоков, объединение результатов и ограничения вычислительных ресурсов не масштабируются идеально; при p выше 12 логических потоков возникает также переподписка ресурсов.

9.6 Эффективность относительно идеального ускорения

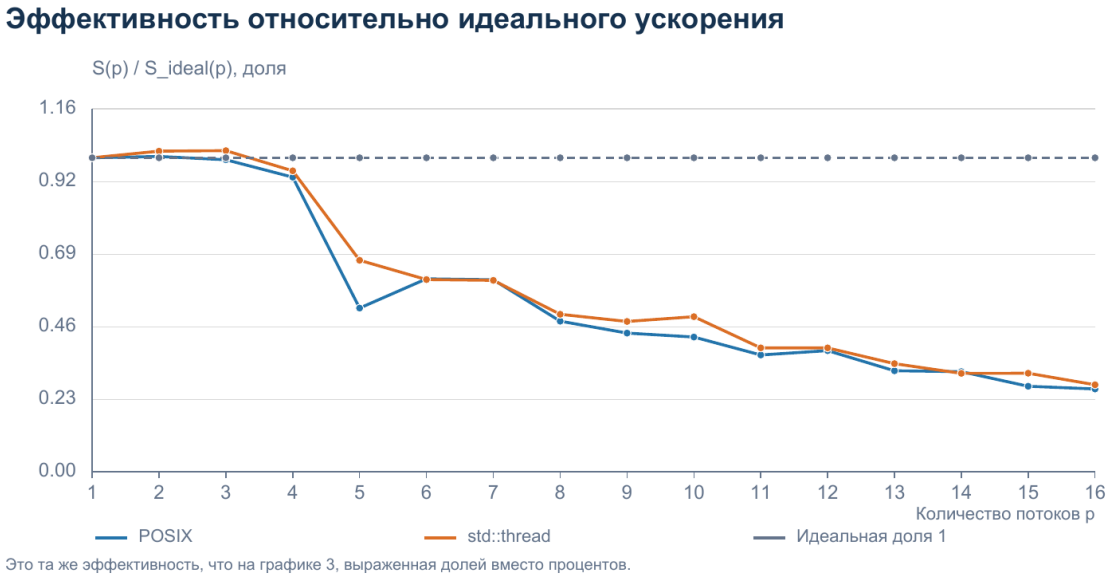


Рисунок 7. Эффективность относительно идеального ускорения.

Доля S/S_ideal = S/p показывает, какую часть идеального ускорения даёт конфигурация. Этот показатель математически совпадает с эффективностью из графика 3, выраженной долей вместо процентов; оба графика приведены отдельно согласно заданию.

10 Демонстрация программы

Программа запускается без аргументов для интерактивного ввода N, числа потоков основного вычисления и максимального числа потоков. Для точного повторения эксперимента использован эквивалентный запуск с аргументами:

```
.\x64\Release\L1pp.exe 20000000 8 16 7 results
```

Ниже приведены реальные скриншоты браузерного просмотра stdout из results/console.txt. Это представление фактического вывода программы; окно терминала на изображениях не имитируется.

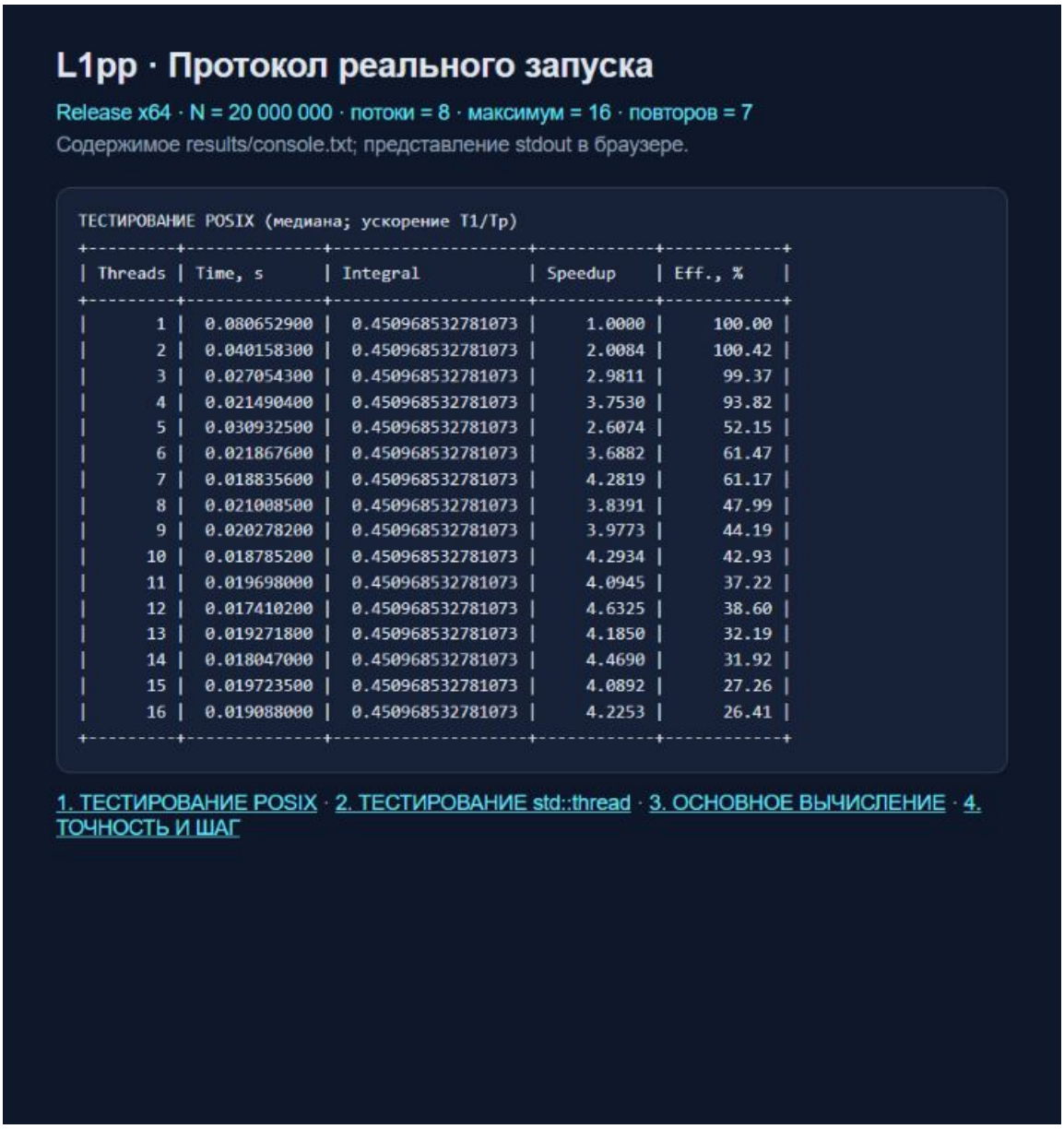


Рисунок 8. Таблица POSIX из протокола запуска.

10 Демонстрация программы продолжение

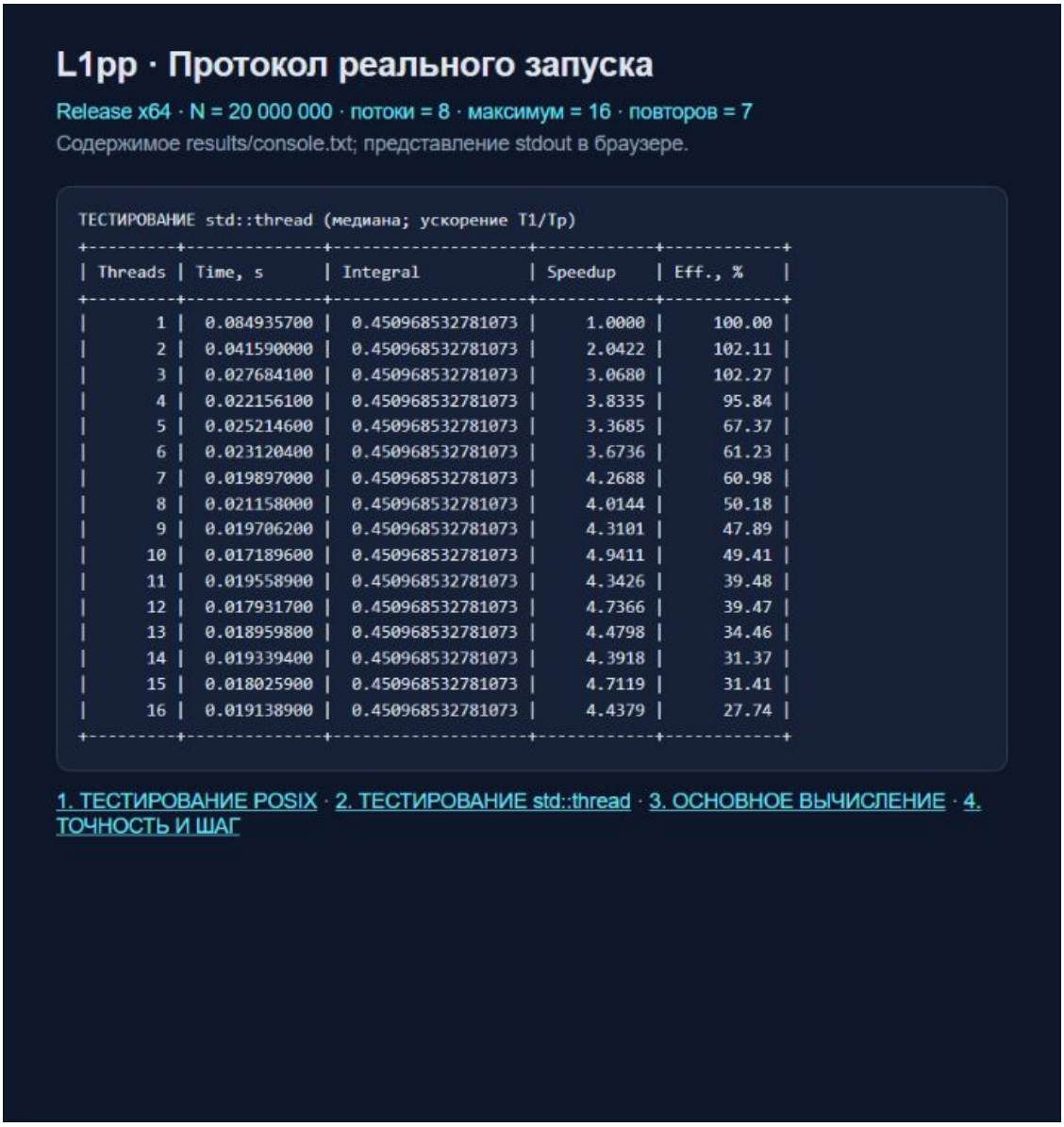


Рисунок 9. Таблица std::thread из того же запуска.

Поля Time, s, Integral, Speedup и Eff., % соответствуют времени, интегралу, ускорению и эффективности. Отдельная таблица следующей страницы сравнивает p = 8 с прямым однопоточным вычислением.

10 Основное вычисление и точность на экране

L1pp · Протокол реального запуска

Release x64 · N = 20 000 000 · потоки = 8 · максимум = 16 · повторов = 7

Содержимое results/console.txt; представление stdout в браузере.

ОСНОВНОЕ ВЫЧИСЛЕНИЕ (ускорение относительно прямого последовательного цикла)

Technology	Threads	Time, s	Integral	Tseq/Tp
direct	1	0.079467800	0.450968532781073	1.0000
POSIX	8	0.021008500	0.450968532781073	3.7826
std::thread	8	0.021158000	0.450968532781073	3.7559

1. ТЕСТИРОВАНИЕ POSIX · 2. ТЕСТИРОВАНИЕ std::thread · 3. ОСНОВНОЕ ВЫЧИСЛЕНИЕ · 4. ТОЧНОСТЬ И ШАГ

Рисунок 10. Основное вычисление.

L1pp · Протокол реального запуска

Release x64 · N = 20 000 000 · потоки = 8 · максимум = 16 · повторов = 7

Содержимое results/console.txt; представление stdout в браузере.

ТОЧНОСТЬ И ШАГ (метод первого порядка, O(h))

N	h	Integral	Abs. error	Runge
10	3.00000e-01	0.417641634018271	3.33269e-02	1.62262e-02
100	3.00000e-02	0.447477943388609	3.49061e-03	1.74089e-03
1000	3.00000e-03	0.450617901685979	3.50649e-04	1.75280e-04
10000	3.00000e-04	0.450933469578366	3.50807e-05	1.75399e-05
100000	3.00000e-05	0.450965042089127	3.50823e-06	1.75411e-06
1000000	3.00000e-06	0.450968199497419	3.50825e-07	1.75412e-07
20000000	1.50000e-07	0.450968532781073	1.75413e-08	8.77063e-09

Runge оценивает ошибку уточнённого I(2N).

Эталон: 0.4509685503223266

Граница ошибки: 5.8007812500000000e-08

Для точности eps: N >= ceil(1.16015625/eps), h=3/N.

CSV сохранены: results

1. ТЕСТИРОВАНИЕ POSIX · 2. ТЕСТИРОВАНИЕ std::thread · 3. ОСНОВНОЕ ВЫЧИСЛЕНИЕ · 4. ТОЧНОСТЬ И ШАГ

Рисунок 11. Сходимость, эталон, граница ошибки и шаг.

11 Выводы

Эффективность параллелизации

Разделение суммы даёт ускорение до 4.63 для POSIX и 4.94 для std::thread относительно одного рабочего потока. После нескольких потоков рост становится неравномерным, а эффективность падает, поэтому увеличение p само по себе не гарантирует меньшего времени.

Сравнение технологий

При одинаковых восьми потоках POSIX занял 0.0210085 с, std::thread — 0.0211580 с; различие медиан составляет 0.71 %. На фоне разброса это не позволяет объявить один API устойчиво быстрее; std::thread удобнее для типизированной передачи аргументов, а POSIX явно возвращает указатель через join.

Закон Амдала

Модель $S(p) = 1 / (s + (1-s)/p)$ ограничивает ускорение при наличии последовательной доли s. Измеренное насыщение согласуется с последовательными затратами и накладными расходами, но неоднородность ресурсов, частота и планировщик нарушают предпосылки простейшей модели, поэтому по одной кривой нельзя надёжно оценить постоянную долю s.

Точность вычислений

Ошибка правых прямоугольников при N = 20,000,000 равна примерно 1.754e-08, что меньше границы 5.801e-08; удвоение N асимптотически уменьшает ошибку вдвое. Изменение числа потоков даёт различия не более $5{,}55 \cdot 10^{-17}$, поэтому точность здесь определяется шагом интегрирования, а не API.

Оптимальная конфигурация

По минимальной медиане данного эксперимента оптимальны 12 потоков POSIX и 10 потоков std::thread. Это локальный результат для N = 20,000,000 и текущей нагрузки: для устойчивой рекомендации следует повторить измерения в сопоставимых условиях и сопоставить межквартильные интервалы.

Практическая значимость

Задачи с независимыми слагаемыми удобно распараллеливать через локальные суммы и последующее объединение без блокировок на каждом узле. Полученный подход применим к квадратурам и другим вычислительным суммам, а выбор N по оценке ошибки позволяет не тратить время на избыточную точность.

12 Воспроизведение и источники

Сборка и запуск

```
PowerShell, из корня проекта:
.\build.ps1
.\x64\Release\L1pp.exe

Воспроизведение замеров:
.\run-experiments.ps1

Проверки:
.\x64\Release\L1pp.exe --self-test
python scripts/verify.py
python scripts/make_report.py
```

Сборка в Visual Studio: открыть L1pp.slnx, выбрать Release и x64. В проекте настроены include, import library и копирование pthreadVC2.dll рядом с исполняемым файлом. Для генератора отчёта нужны Python, ReportLab, Pillow и Poppler; пути к Poppler можно задать переменной CODEX_PDFTOPPM.

Исходники и GitHub

Полный текст C++ приведён в приложении и находится в папке L1pp. Репозиторий GitHub отсутствует: по прямому указанию заказчика создавать его не требуется, поэтому внешняя ссылка не приводится. Все файлы для последующего размещения включены в комплект работы.

Использованные материалы

1. Условие лабораторной работы и предоставленный скриншот варианта.

2. Pthreads-win32 2.9.1: pthread/Pre-built.2/README, include/pthread.h; комплект библиотек x64 из предоставленной папки pthread.

3. Рабочий проект стандарта C++, конструктор std::thread: eel.is/c++draft/thread.thread.constr. Конструктор принимает вызываемый объект и отдельные аргументы; обычное возвращаемое значение функции потока игнорируется.

4. Собственные экспериментальные данные: results/raw.csv, summary.csv, main.csv, accuracy.csv; проверка — results/verification.json.

Материалы POSIX Threads for Win.pdf и 2026_ПП_Л1.pdf в предоставленной папке отсутствуют. Они не включены в список использованных источников; свойства библиотеки проверены по её локальным заголовкам и README.

Приложение Текст программы

integration.hpp

```
#pragma once
#include <cstdint>
namespace lab {
using Index = std::uint64_t;
constexpr double A = 2.0, B = 5.0;
constexpr double derivative_bound = 33.0 / 128.0;
double function(double x);
double partial_sum(Index first, Index last, double a, double h);
double sequential(Index n);
double posix_integral(Index n, unsigned threads);
double std_integral(Index n, unsigned threads);
double reference_integral();
void validate(Index n, unsigned threads);
void range(Index n, unsigned threads, unsigned id, Index& first, Index& last);
}
```

Приложение Текст программы

integration.cpp

```
#include "integration.hpp"
#include <cmath>
#include <stdexcept>
namespace lab {
double function(double x) {
    return (1.0 + x * x) / (x * x * x * std::sqrt(2.0 + x));
}
double partial_sum(Index first, Index last, double a, double h) {
    // Kahan summation limits roundoff without changing the quadrature rule.
    double sum = 0.0, correction = 0.0;
    for (Index i = first; i < last; ++i) {
        const double value = function(a + static_cast<double>(i + 1) * h);
        const double y = value - correction;
        const double next = sum + y;
        correction = (next - sum) - y;
        sum = next;
    }
    return sum;
}
void validate(Index n, unsigned threads) {
    // Internal limit also permits the 2N accuracy check for user N <= 10^12.
    if (n == 0 || n > 2000000000000ULL)
        throw std::invalid_argument("Internal N must be in [1, 2*10^12]");
    if (threads == 0 || threads > 256 || threads > n)
        throw std::invalid_argument("Threads must be in [1, min(N,256)]");
}
void range(Index n, unsigned threads, unsigned id, Index& first, Index& last) {
    // Distribute the remainder: block sizes differ by at most one.
    const Index q = n / threads, r = n % threads;
    first = id * q + (id < r ? id : r);
    last = first + q + (id < r ? 1 : 0);
}
double sequential(Index n) {
    validate(n, 1);
    const double h = (B - A) / static_cast<double>(n);
    return h * partial_sum(0, n, A, h);
}
double reference_integral() {
    const auto primitive = [](double x) {
        const double t = std::sqrt(x + 2.0), r = std::sqrt(2.0);
        return 35.0 / (32.0 * r) * std::log((t - r) / (t + r))
             + t * (3.0 / (16.0 * x) - 1.0 / (4.0 * x * x));
    };
    return primitive(B) - primitive(A);
}
}
```

Приложение Текст программы

posix.cpp

```
#include "integration.hpp"
#if defined(_MSC_VER) && _MSC_VER >= 1900
#include <time.h>
#define HAVE_STRUCT_TIMESPEC
#endif
#include <pthread.h>
#include <iostream>
#include <memory>
#include <new>
#include <stdexcept>
#include <string>
#include <vector>
namespace lab {
namespace {
struct Arguments { Index first, last; double a, h; };
void* worker(void* raw) {
    const auto& args = *static_cast<Arguments*>(raw);
    // Heap storage stays valid after this worker returns.
    auto* result = new (std::nothrow) double;
    if (result) *result = partial_sum(args.first, args.last, args.a, args.h);
    return result; // Received by the caller ONLY through pthread_join.
}
double join_result(pthread_t thread) {
    void* raw = nullptr;
    const int code = pthread_join(thread, &raw);
    if (code != 0) {
        // Do not unwind storage possibly still used by a live worker.
        std::cerr << "pthread_join failed: " << code << '\n';
        std::terminate();
    }
    std::unique_ptr<double> value(static_cast<double*>(raw));
    if (!value) throw std::bad_alloc();
    return *value;
}
}
double posix_integral(Index n, unsigned threads) {
    validate(n, threads);
    const double h = (B - A) / static_cast<double>(n);
    std::vector<pthread_t> handles(threads);
    std::vector<Arguments> args(threads);
    unsigned created = 0;
    int create_error = 0;
    for (; created < threads; ++created) {
        range(n, threads, created, args[created].first, args[created].last);
        args[created].a = A; args[created].h = h;
        create_error = pthread_create(&handles[created], nullptr, worker, &args[created]);
        if (create_error != 0) break;
    }
    double sum = 0.0, correction = 0.0;
    bool allocation_failed = false;
    for (unsigned t = 0; t < created; ++t) {
        try {
            const double y = join_result(handles[t]) - correction;
            const double next = sum + y;
            correction = (next - sum) - y; sum = next;
        } catch (const std::bad_alloc&) {
            allocation_failed = true; // Join all other workers before throwing.
        }
    }
    if (create_error)
        throw std::runtime_error("pthread_create failed: " + std::to_string(create_error));
    if (allocation_failed) throw std::bad_alloc();
    return sum * h;
}
}
```

Приложение Текст программы

std_thread.cpp

```
#include "integration.hpp"
#include <functional>
#include <thread>
#include <vector>
namespace lab {
namespace {
// Parameters are passed separately, never bundled in a structure.
void worker(Index first, Index last, double a, double h, double& result) {
    result = partial_sum(first, last, a, h);
}
}
double std_integral(Index n, unsigned threads) {
    validate(n, threads);
    const double h = (B - A) / static_cast<double>(n);
    std::vector<double> results(threads);
    std::vector<std::thread> handles;
    handles.reserve(threads);
    try {
        for (unsigned t = 0; t < threads; ++t) {
            Index first, last;
            range(n, threads, t, first, last);
            handles.emplace_back(worker, first, last, A, h, std::ref(results[t]));
        }
    } catch (...) {
        for (auto& thread : handles) thread.join();
        throw;
    }
    for (auto& thread : handles) thread.join();
    double sum = 0.0, correction = 0.0;
    for (double value : results) {
        const double y = value - correction;
        const double next = sum + y;
        correction = (next - sum) - y; sum = next;
    }
    return h * sum;
}
}
```

Приложение Текст программы

main.cpp

```
#include "integration.hpp"
#include <algorithm>
#include <chrono>
#include <cmath>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <random>
#include <stdexcept>
#include <string>
#include <thread>
#include <vector>
#ifdef _WIN32
#include <windows.h>
#endif

namespace {
using lab::Index;
using Clock = std::chrono::steady_clock;
using Integrator = double (*)(Index, unsigned);
struct Sample { double seconds, value; };
struct Row { int api; unsigned threads; double seconds, value, speedup, efficiency; };

Index number(const std::string& text) {
    if (text.empty() || text.find_first_not_of("0123456789") != std::string::npos)
        throw std::invalid_argument("Expected a positive integer: " + text);
    const Index value = std::stoull(text);
    if (value == 0) throw std::invalid_argument("Expected a positive integer");
    return value;
}
Index input(const char* prompt) {
    std::cout << prompt << std::flush;
    std::string line;
    if (!std::getline(std::cin, line)) throw std::runtime_error("Input ended");
    return number(line);
}
Sample timed(Integrator integrate, Index n, unsigned p) {
    const auto start = Clock::now();
    const double value = integrate(n, p);
    const auto stop = Clock::now();
    return {std::chrono::duration<double>(stop - start).count(), value};
}
double serial_adapter(Index n, unsigned) { return lab::sequential(n); }
Sample median(std::vector<Sample> values) {
    std::sort(values.begin(), values.end(), [](Sample a, Sample b) {
        return a.seconds < b.seconds;
    });
    return values[values.size() / 2];
}
void line() {
    std::cout << "+---------+--------------+--------------------+------------+------------+\n";
}
void table(const std::string& title, const std::vector<Row>& rows, int api) {
    std::cout << '\n' << title << '\n';
    line();
    std::cout << "| Threads | Time, s      | Integral           | Speedup    | Eff., %    |\n";
    line();
    for (const auto& r : rows) if (r.api == api) {
        std::cout << "| " << std::setw(7) << r.threads << " | "
                  << std::fixed << std::setprecision(9) << std::setw(12) << r.seconds << " | "
                  << std::setprecision(15) << std::setw(18) << r.value << " | "
                  << std::setprecision(4) << std::setw(10) << r.speedup << " | "
                  << std::setprecision(2) << std::setw(10) << r.efficiency << " |\n";
    }
    line();
}
void self_test() {
    const double ref = lab::reference_integral();
    for (Index n : {Index(1), Index(2), Index(7), Index(31), Index(1000), Index(10000)}) {
        const double serial = lab::sequential(n);
        if (!(serial < ref && ref - serial <= 1.16015625 / static_cast<double>(n)))
            throw std::runtime_error("Quadrature bound failed");
        for (unsigned p = 1; p <= std::min<Index>(n, 8); ++p) {
```

```
            Index previous = 0;
            for (unsigned t = 0; t < p; ++t) {
                Index first, last;
                lab::range(n, p, t, first, last);
                if (first != previous || last <= first)
                    throw std::runtime_error("Partition failed");
                previous = last;
            }
            if (previous != n) throw std::runtime_error("Coverage failed");
            for (auto fn : {lab::posix_integral, lab::std_integral})
                if (std::abs(fn(n, p) - serial) > 2e-14)
                    throw std::runtime_error("Parallel sum failed");
        }
    }
    const double e1 = ref - lab::sequential(1000), e2 = ref - lab::sequential(2000);
    if (e1 / e2 < 1.99 || e1 / e2 > 2.01) throw std::runtime_error("Order test failed");
    for (auto pair : {std::pair<Index,unsigned>{0,1}, {1,0}, {2,3}, {1000,257}}) {
        bool rejected = false;
        try { lab::validate(pair.first, pair.second); }
        catch (const std::invalid_argument&) { rejected = true; }
        if (!rejected) throw std::runtime_error("Validation failed");
    }
    std::cout << "PASS: partition, POSIX, std::thread, error bound, convergence, invalid inputs.\n";
}
}

int main(int argc, char** argv) {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
#endif
    try {
        if (argc == 2 && std::string(argv[1]) == "--self-test") { self_test(); return 0; }
        if (argc == 2 && std::string(argv[1]) == "--help") {
            std::cout << "L1pp [N main_threads max_threads [odd_repeats [output_dir]]]\n"
                         "No arguments: interactive input. --self-test: correctness checks.\n";
            return 0;
        }
        if (argc != 1 && (argc < 4 || argc > 6)) throw std::invalid_argument("Use --help");
        std::cout << "ЛАБОРАТОРНАЯ РАБОТА 1. ПРАВЫЕ ПРЯМОУГОЛЬНИКИ\n"
                     "f(x) = (1+x^2)/(x^3*sqrt(2+x)), [2,5]\n";
        const Index n = argc > 1 ? number(argv[1]) : input("Количество разбиений N: ");
        if (n > 1000000000000ULL) throw std::invalid_argument("N must be in [1, 10^12]");
        const Index main_input = argc > 1 ? number(argv[2]) : input("Потоки основного вычисления: ");
        const Index max_input = argc > 1 ? number(argv[3]) : input("Максимум потоков для тестирования:
");
        const Index repeats_input = argc > 4 ? number(argv[4]) : 5;
        if (main_input > 256 || max_input > 256)
            throw std::invalid_argument("Maximum allowed threads: 256");
        if (repeats_input > 99 || repeats_input % 2 == 0)
            throw std::invalid_argument("Repeats must be odd and in [1,99]");
        const unsigned main_p = static_cast<unsigned>(main_input);
        const unsigned max_p = static_cast<unsigned>(max_input);
        const unsigned repeats = static_cast<unsigned>(repeats_input);
        lab::validate(n, main_p); lab::validate(n, max_p);
        const std::filesystem::path output = argc > 5 ? argv[5] : "results";
        std::filesystem::create_directories(output);
        std::ofstream raw(output / "raw.csv"), summary(output / "summary.csv"),
                      main_csv(output / "main.csv"), accuracy(output / "accuracy.csv"),
                      metadata(output / "metadata.txt");
        if (!raw || !summary || !main_csv || !accuracy || !metadata)
            throw std::runtime_error("Cannot create output files");
        raw << "technology,threads,repeat,seconds,result\n" << std::setprecision(17);
        summary << "technology,threads,seconds,result,speedup,efficiency_percent\n"
                << std::setprecision(17);
        main_csv << "technology,threads,seconds,result,speedup_vs_direct\n"
                 << std::setprecision(17);
        accuracy << "n,h,result,absolute_error,relative_error,runge_estimate,bound,observed_order\n"
                 << std::setprecision(17);
        const double h = 3.0 / static_cast<double>(n);
        const double reference = lab::reference_integral();
        std::cout << "\nN=" << n << "; h=" << std::scientific << h << "; main=" << main_p
                  << "; max=" << max_p << "; repeats=" << repeats << "; hardware="
                  << std::thread::hardware_concurrency() << "\n";
        metadata << "N=" << n << "\nmain_threads=" << main_p << "\nmax_threads=" << max_p
                 << "\nrepeats=" << repeats << "\nhardware_threads="
                 << std::thread::hardware_concurrency()
                 << "\nclock=steady_clock\nstatistic=median\nwarmup=one_per_configuration\n"
                 << "shuffle_seed=412\ntiming=allocation+creation+computation+join+reduction\n";
        Integrator functions[] = {lab::posix_integral, lab::std_integral};
```

```
        const char* names[] = {"POSIX", "std::thread"};
        for (auto fn : functions)
            for (unsigned p = 1; p <= max_p; ++p) (void)fn(n, p);
        (void)lab::sequential(n);
        std::vector<Sample> serial_samples;
        std::vector<std::vector<std::vector<Sample>>> samples(2,
            std::vector<std::vector<Sample>>(max_p + 1));
        std::vector<std::pair<int, unsigned>> schedule;
        for (int api = 0; api < 2; ++api)
            for (unsigned p = 1; p <= max_p; ++p) schedule.emplace_back(api, p);
        std::mt19937 generator(412);
        for (unsigned repeat = 1; repeat <= repeats; ++repeat) {
            std::shuffle(schedule.begin(), schedule.end(), generator);
            const auto direct = timed(serial_adapter, n, 1);
            serial_samples.push_back(direct);
            raw << "direct,1," << repeat << ',' << direct.seconds << ',' << direct.value << '\n';
            for (auto config : schedule) {
                const auto s = timed(functions[config.first], n, config.second);
                samples[config.first][config.second].push_back(s);
                raw << names[config.first] << ',' << config.second << ',' << repeat
                    << ',' << s.seconds << ',' << s.value << '\n';
            }
            std::cout << "Повтор " << repeat << '/' << repeats << " завершён.\n" << std::flush;
        }
        const auto direct = median(serial_samples);
        std::vector<Row> rows;
        for (int api = 0; api < 2; ++api) {
            const double baseline = median(samples[api][1]).seconds;
            for (unsigned p = 1; p <= max_p; ++p) {
                const auto s = median(samples[api][p]);
                const double speed = baseline / s.seconds, eff = 100.0 * speed / p;
                rows.push_back({api, p, s.seconds, s.value, speed, eff});
                summary << names[api] << ',' << p << ',' << s.seconds << ',' << s.value
                        << ',' << speed << ',' << eff << '\n';
            }
            table(std::string("ТЕСТИРОВАНИЕ ") + names[api] + " (медиана; ускорение T1/Tp)", rows, api);
        }
        std::cout << "\nОСНОВНОЕ ВЫЧИСЛЕНИЕ (ускорение относительно прямого последовательного цикла)\n"
                     "+--------------+---------+--------------+--------------------+------------+\n"
                     "| Technology   | Threads | Time, s      | Integral           | Tseq/Tp    |\n"
                     "+--------------+---------+--------------+--------------------+------------+\n";
        const auto main_row = [&](const char* name, unsigned p, Sample s) {
            const double speed = direct.seconds / s.seconds;
            std::cout << "| " << std::setw(12) << name << " | " << std::setw(7) << p << " | "
                      << std::fixed << std::setprecision(9) << std::setw(12) << s.seconds << " | "
                      << std::setprecision(15) << std::setw(18) << s.value << " | "
                      << std::setprecision(4) << std::setw(10) << speed << " |\n";
            main_csv << name << ',' << p << ',' << s.seconds << ',' << s.value << ',' << speed << '\n';
        };
        main_row("direct", 1, direct);
        for (int api = 0; api < 2; ++api) {
            Sample s{};
            if (main_p <= max_p) s = median(samples[api][main_p]);
            else {
                (void)functions[api](n, main_p);
                std::vector<Sample> extra;
                for (unsigned r = 1; r <= repeats; ++r) {
                    const auto run = timed(functions[api], n, main_p);
                    extra.push_back(run);
                    raw << names[api] << ',' << main_p << ',' << r << ','
                        << run.seconds << ',' << run.value << '\n';
                }
                s = median(extra);
            }
            main_row(names[api], main_p, s);
        }
        std::cout << "+--------------+---------+--------------+--------------------+------------+\n";
        std::cout << "\nТОЧНОСТЬ И ШАГ (метод первого порядка, O(h))\n"

    "+--------------+--------------+--------------------+--------------+--------------+\n"
                     "| N            | h            | Integral           | Abs. error   | Runge
    |\n"

    "+--------------+--------------+--------------------+--------------+--------------+\n";
        std::vector<Index> sizes = {10, 100, 1000, 10000, 100000, 1000000, n};
        std::sort(sizes.begin(), sizes.end());
        sizes.erase(std::unique(sizes.begin(), sizes.end()), sizes.end());
        for (Index size : sizes) {
            const double coarse = lab::sequential(size), fine = lab::sequential(2 * size);
            const double err = std::abs(reference - coarse);
```

```
            const double fine_err = std::abs(reference - fine);
            const double runge = std::abs(fine - coarse); // estimate for I_(2N), order 1
            accuracy << size << ',' << 3.0 / static_cast<double>(size) << ',' << coarse << ',' << err <<
 ','
                     << err / std::abs(reference) << ',' << runge << ','
                     << 1.16015625 / static_cast<double>(size) << ',' << std::log2(err / fine_err) <<
'\n';
            std::cout << "| " << std::setw(12) << size << " | " << std::scientific <<
std::setprecision(5)
                      << std::setw(12) << 3.0 / static_cast<double>(size) << " | " << std::fixed
                      << std::setprecision(15) << std::setw(18) << coarse << " | " << std::scientific
                      << std::setprecision(5) << std::setw(12) << err << " | "
                      << std::setw(12) << runge << " |\n";
        }
        std::cout <<
"+--------------+--------------+--------------------+--------------+--------------+\n"
                  << "Runge оценивает ошибку уточнённого I(2N).\nЭталон: " << std::fixed
                  << std::setprecision(16) << reference << "\nГраница ошибки: " << std::scientific
                  << 1.16015625 / static_cast<double>(n)
                  << "\nДля точности eps: N >= ceil(1.16015625/eps), h=3/N.\nCSV сохранены: "
                  << output.string() << '\n';
        return 0;
    } catch (const std::exception& e) {
        std::cerr << "ERROR: " << e.what() << '\n';
        return 1;
    }
}
```